

Lecture 2 - Computer Organization and Architecture

Overview

This lecture introduces the basics of **computer organization and architecture**, including the **computer hierarchy**, **Von Neumann model**, **instruction cycle**, and **memory organization**. It also covers **addressing modes**, the **programming process**, common **errors**, and **algorithm design**, providing a solid foundation for understanding how computers work internally.

Topics

- | | |
|--|---|
| 1. Computer Organization vs Computer Architecture | 4. Instruction Cycle |
| 2. Computer Level Hierarchy | 5. Memory Organization & Addressing Modes |
| 3. Von Neumann Architecture | 6. Programming Process, Errors, and Algorithms |

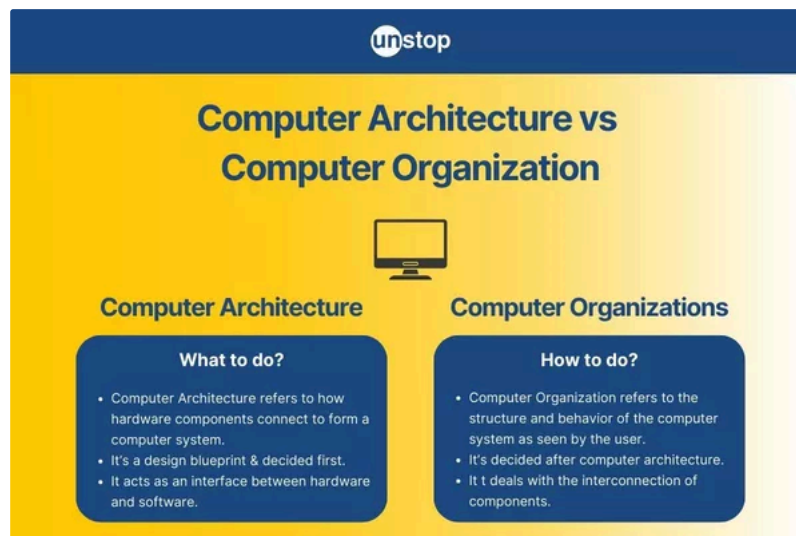
Objectives / Outcome

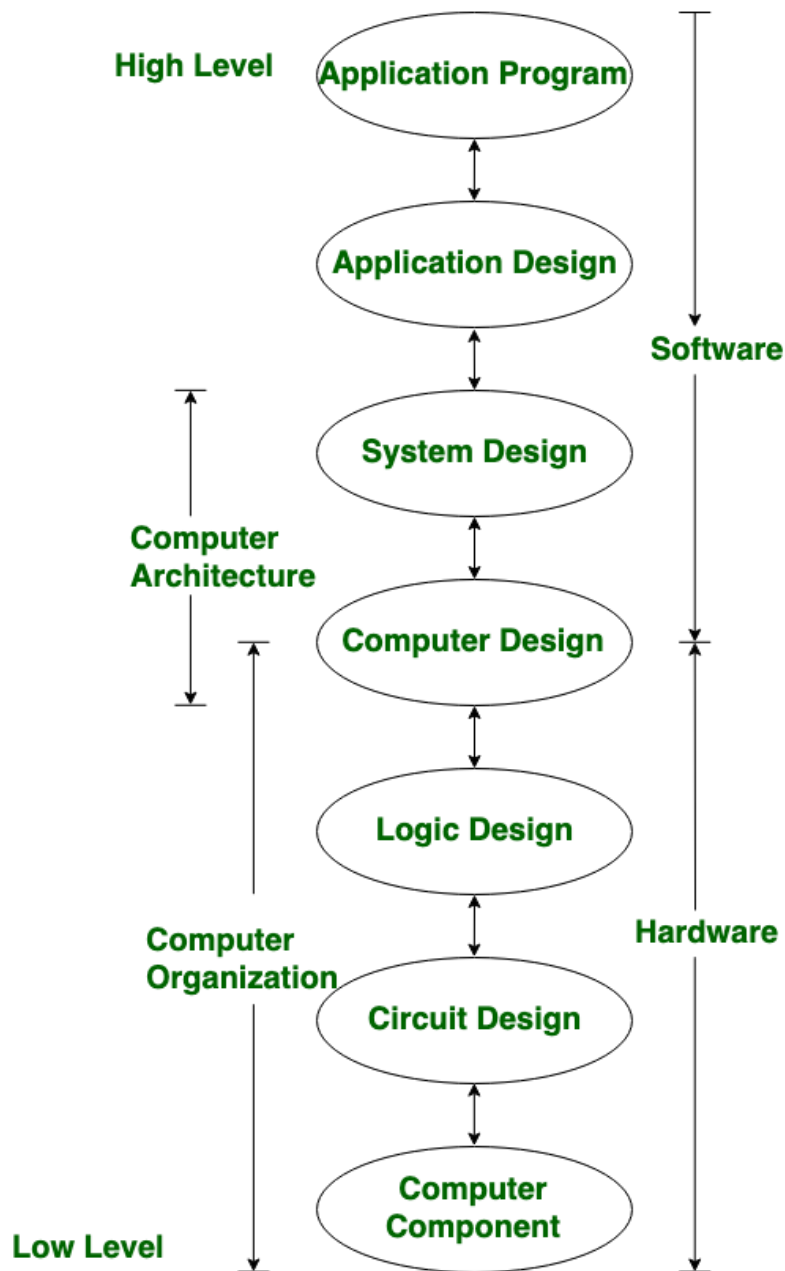
1. Understand the difference between **computer organization** and **computer architecture**.
2. Describe the **levels of the computer hierarchy**.
3. Explain the components and principles of the **Von Neumann architecture**.
4. Understand the **instruction cycle** and its major stages.
5. Explain **memory organization** and different **addressing modes**.
6. Understand the **programming process**, **error types**, and **algorithm design**.

Computer Organization and Architecture

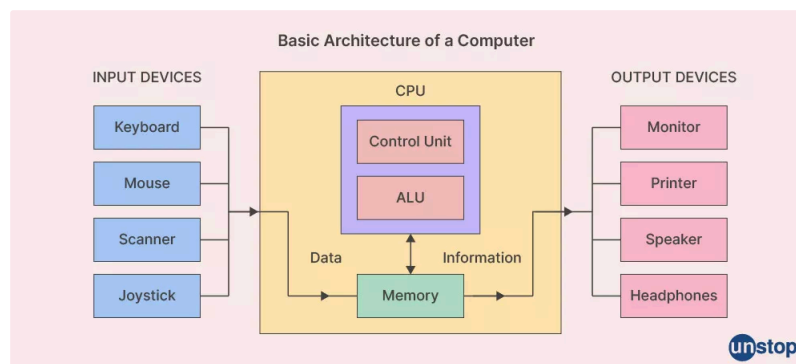
Definition

- **Computer Organization:** Refers to the operational units and their interconnections that realize the architectural specifications.
- **Computer Architecture:** Refers to the attributes of a system visible to a programmer or, put another way, those attributes that have a direct impact on the logical execution of a program.





Computer System

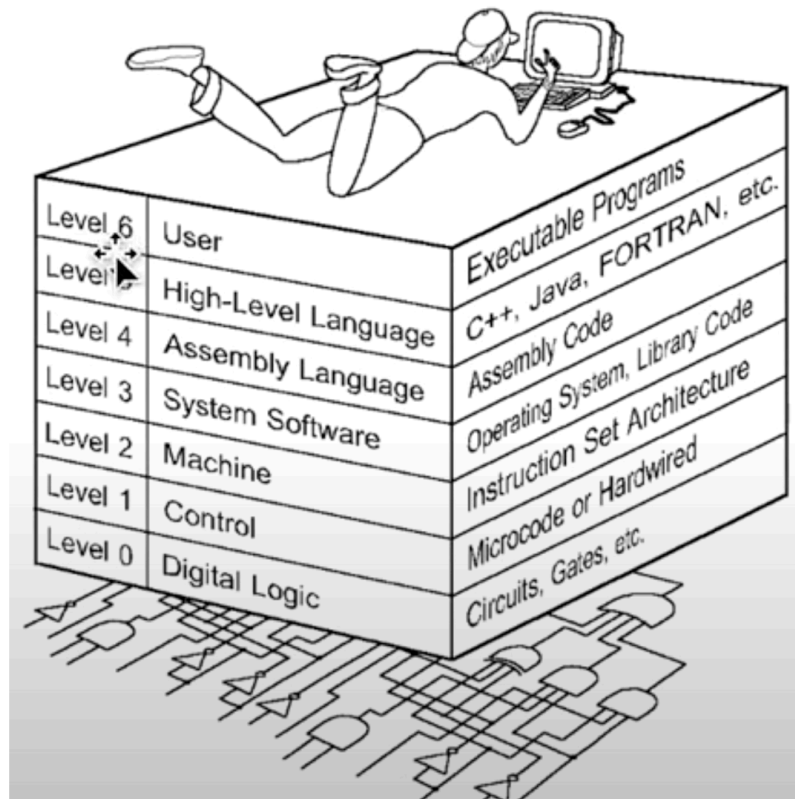


Example Question

- Q: What is the difference between computer organization and computer architecture?
- A: Computer organization deals with the physical components and their interconnections, while computer architecture refers to the logical aspects of a system that affect its performance and functionality from the programmer's perspective.

Computer Level Hierarchy

Levels of Computer Hierarchy



- **Level 0:** Digital Logic Level (gates, circuits)
- **Level 1:** Microarchitecture Level (registers, ALU)
- **Level 2:** Instruction Set Architecture (ISA) Level (machine language)
- **Level 3:** Operating System Level
- **Level 4:** Assembly Language Level
- **Level 5:** High-Level Language Level
- **Level 6:** User Level

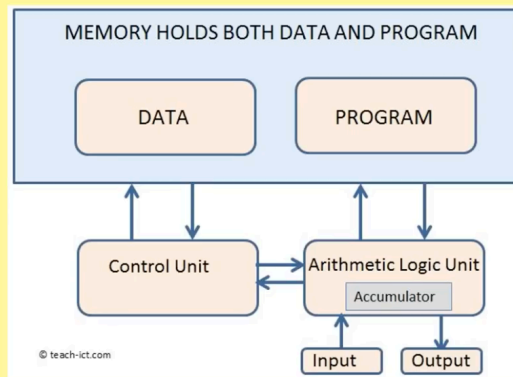
Example Question

- **Q:** Describe the instruction set architecture (ISA) level in the computer level hierarchy.
A: The ISA level defines the machine language instructions that a computer can execute, including the types of instructions, addressing modes, and hardware support for data types.

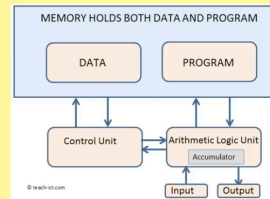
Von Neumann Architecture

Von-Neumann computer architecture design was proposed in 1945. It was later known as **Von-Neumann architecture**.

Von Neumann Layout



Von Neumann Layout



- **Control Unit**

- Responsible for decoding the instructions and controlling how data moves around the computer system.

- **ALU**

- Carries out the calculations and logical decisions required by the program instruction (for example addition, subtraction and comparisons such as equal to, greater than or less than).

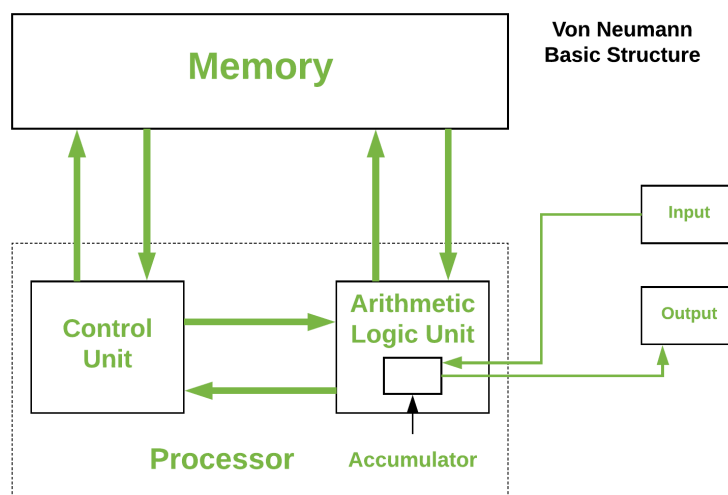
- **Bus**

- The wires that carry data around a computer.

Historically there have been two types of computers:

1. **Fixed Program Computers:** Their function is very specific and they couldn't be reprogrammed, e.g., calculators.
2. **Stored Program Computers:** These can be programmed to carry out many different tasks; applications are stored on them, hence the name.

Modern computers are based on a **stored-program concept** introduced by John Von Neumann. In this concept, **programs and data are stored in the same memory**. This makes a computer built with this architecture much easier to reprogram.



It is also known as an **ISA (Instruction Set Architecture) computer** and has three basic units:

- Central Processing Unit (CPU)
- Main Memory Unit
- Input/Output Devices

Central Processing Unit (CPU)

The CPU is an electronic circuit that executes instructions of a computer program. It has the following major components:

- **Control Unit (CU):** Handles processor control signals, directs input/output flow, fetches instructions, and controls data movement.
- **Arithmetic and Logic Unit (ALU):** Performs arithmetic (addition, subtraction) and logical operations, comparisons, and bit shifting.
- **Registers:** Small storage locations inside the CPU for temporary data storage.

Components of Von Neumann Architecture

- **Central Processing Unit (CPU):** Executes instructions.
- **Memory:** Stores data and instructions.
- **Input/Output Devices:** Allow interaction with the computer.
- **Bus:** Data path that connects CPU, memory, and I/O devices.

Key Concepts

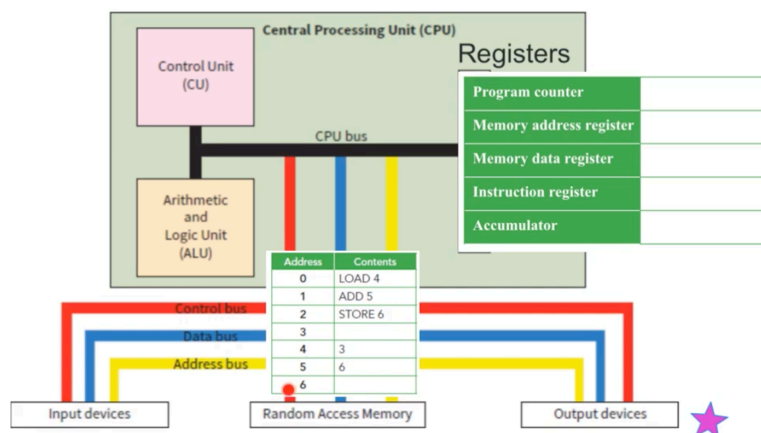
- **Stored Program Concept:** Instructions and data are stored in memory.
- **Sequential Execution:** Instructions are processed one after another.

Example Question

- **Q:** What is the stored program concept in the Von Neumann architecture?
A: The stored program concept means that instructions to be executed by the computer are represented as binary values and stored in memory.

Simple Computer Instruction Processing

Instruction Cycle



1. **Fetch:** Retrieve an instruction from memory.
2. **Decode:** Determine what the instruction means.
3. **Execute:** Perform the instruction.
4. **Store:** Write the result back to memory if necessary.

Example Question

- **Q:** What are the four main stages of the instruction cycle?
A: Fetch, Decode, Execute, Store.

Practical Question

- **Q:** Describe the steps involved in fetching an instruction from memory in the Von Neumann architecture.
A: The steps are:
 1. The address of the next instruction is loaded from the program counter into the memory address register.
 2. The instruction is fetched from memory and loaded into the instruction register.
 3. The program counter is incremented to point to the next instruction.

Memory Organization and Addressing

Types of Memory

- **Primary Memory:** RAM, ROM.
- **Secondary Memory:** Hard drives, SSDs.

Addressing Modes

- **Direct Addressing:** The address of the data is given explicitly.
- **Indirect Addressing:** The address of the address of the data is given.
- **Indexed Addressing:** An index register is used to modify the address.

Example Question

- **Q:** Explain the difference between primary and secondary memory.
A: Primary memory is directly accessible by the CPU and includes RAM and ROM. Secondary memory is used for long-term storage and includes devices like hard drives and SSDs.

Practical Question

- **Q:** Given a 32-bit memory address, how many distinct memory locations can be addressed?
A: $2^{32} = 4,294,967,296$ memory locations.

The Programming Process, Errors, and Methodology

Steps in Programming

- **Problem Definition:** Understand the problem.
- **Algorithm Design:** Plan the solution.
- **Coding:** Write the program.
- **Testing:** Find and fix errors.
- **Documentation:** Explain the code and how it works.

Types of Errors

- **Syntax Errors:** Mistakes in the code's grammar.
- **Runtime Errors:** Errors that occur while the program is running.
- **Logic Errors:** Mistakes in the algorithm's logic.

Example Question

- **Q:** What are the main types of programming errors?
A: Syntax errors, runtime errors, and logic errors.

Algorithm Design

Definition

An **algorithm** is a step-by-step procedure for solving a problem.

Characteristics of a Good Algorithm

- **Finiteness:** The algorithm terminates after a finite number of steps.
- **Definiteness:** Each step is precisely defined.
- **Input/Output:** The algorithm has specified inputs and outputs.
- **Effectiveness:** The steps are basic enough to be carried out.

Example Algorithm

Problem: Find the maximum number in a list of integers.

Algorithm:

1. Initialize **max** to the first element of the list.
2. For each element in the list, if the element is greater than **max**, set **max** to the element.
3. Return **max**.

Example Question

- **Q:** What are the key characteristics of a good algorithm?
A: Finiteness, definiteness, input/output, and effectiveness.